

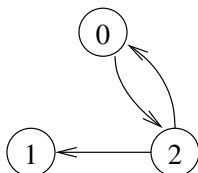
Programmation Fonctionnelle

TD n° 7 : Records et graphes

On considère les graphes orientés dont les sommets sont étiquetés par des segments initiaux d'entiers naturels. On représente ces graphes en OCaml par des records ayant un champ `size:int` et un champ `fwd : int -> (int list)` :

```
type transitions = int -> (int list)
type graph = {size : int; fwd : transitions}
```

La valeur du champ `fwd` d'un graphe est appelée sa *fonction de transition*. Si $\{size = n; fwd = f\}$ représente le graphe G et si p désigne un entier entre 0 et $n-1$, alors $(f\ p)$ renvoie la liste triée et sans répétitions des sommets de G reliés à p par un arc partant de p . Par exemple, le graphe suivant



peut être représenté par $\{size = 3; fwd = f\}$, pour toute fonction f vérifiant :

```
(f 0) -> [2],
(f 1) -> [],
(f 2) -> [0; 1].
```

Le comportement de f sur les entiers supérieurs à 2 est sans importance. Rappelons qu'un chemin de longueur k dans un graphe est une suite de sommets $(p_0, \dots, p_k)$ tel que pour chaque i , il existe un arc de p_i vers p_{i+1} . Par exemple, dans le graphe ci-dessus, (1) est un chemin de longueur 0 partant de 1 et atteignant 1, et $(2, 0, 2, 1)$ est un chemin de longueur 3 partant de 2 et atteignant 1.

Exercice 1 : Vous pouvez pour cet exercice utiliser les fonctions du module `List` de OCaml.

1. Noter qu'un chemin est représentable par une liste d'entiers non vide. Ecrire :

```
chemin : transitions -> int list -> bool
```

tel que si f est une fonction de transition et l est une liste non vide de sommets du graphe associé, alors `chemin f l` vaut `true` si l est un chemin du graphe, et `false` sinon. Par exemple, si f est la fonction de transition du graphe ci-dessus, on doit avoir

```
- chemin f [2; 0; 2; 1] -> true
- chemin f [2; 1; 2] -> false
```

2. Ecrire :

```
atteints : transitions -> int -> int -> int list
```

tel que si f est une fonction de transition et x est un sommet du graphe associé, $(atteints\ f\ k\ x)$ renvoie la liste triée et sans répétitions des sommets atteints par un chemin partant de x et de longueur au plus k . Par exemple, si f est la fonction de transition du graphe ci-dessus, on doit avoir :

- `atteints f 0 0` $\rightarrow$ `[0]`
- `atteints f 1 0` $\rightarrow$ `[0; 2]`
- `atteints f 2 0` $\rightarrow$ `[0; 1; 2]`
- `atteints f 3 0` $\rightarrow$ `[0; 1; 2]`
- etc.

3. A partir de la fonction précédente, écrire :

`accessible : transitions -> int -> int -> bool`

tel que si `f` est une fonction de transition si `x` et `y` sont deux sommets du graphe associé, `accessible f x y` vaut `true` s'il existe un chemin dans le graphe partant de `x` et menant à `y`, et `false` sinon. Par exemple, si `f` représente le graphe ci-dessus, on doit avoir :

- `accessible f 0 1` $\rightarrow$ `true`
- `accessible f 1 0` $\rightarrow$ `false`

Attention aux bouclages.

Exercice 2 : Dans cet exercice, vous allez modifier des graphes en changeant leur fonction de transition.

1. Écrire une fonction

`modify: graph -> int -> int list -> graph`

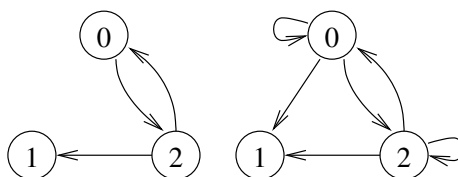
telle que `modify g x l` renvoie un graphe qui a les mêmes sommets que `g` et dans lequel les flèches sortantes du sommet `x` sont celles spécifiées par la liste de sommets `l`. On supposera que `x` et les éléments de `l` sont bien des sommets de `g` et que `l` est triée et sans répétitions. Par exemple, si `g` est le graphe ci-dessus on aura que

```
(modify g 0 [1]).fwd 0 donne [1]
(modify g 0 [1]).fwd 1 donne []
(modify g 0 [1]).fwd 2 donne [0;1]
```

2. Etant donné un graphe G , on appellera 1-complétion de G le graphe G' contenant tous les sommets et flèches de G , ainsi que toutes les flèches ajoutées en respectant la contrainte suivante :

Si G contient une flèche allant de p à q et une flèche allant de q à r , alors G' contient une flèche allant de p à r .

Voici par exemple le graphe précédent et sa 1-complétion. On a $0 \rightarrow 2 \rightarrow 0$, $2 \rightarrow 0 \rightarrow 2$ et $0 \rightarrow 2 \rightarrow 1$ dans le graphe initial, donc $0 \rightarrow 0$, $2 \rightarrow 2$ et $0 \rightarrow 1$ dans sa complétion.



Ecrire une fonction

`un_complétion : graph -> graph`

calculant la 1-complétion d'un graphe.

3. Etant donné un graphe G^0 , on peut calculer sa 1-complétion G^1 , la 1-complétion G^2 de G^1 , la 1-complétion G^3 de G^2 , etc. Il est facile de voir que cette suite finit par atteindre un graphe G^n tel que $G^{n+1} = G^n$: on ne peut pas ajouter indéfiniment des flèches au graphe initial. Le premier G^n tel que $G^{n+1} = G^n$ sera appelé *complétion maximale* de G^0 .

3.1 Écrire une fonction

`equal : graph -> graph -> bool`

qui vérifie si ses deux arguments représentent le même graphe.

3.2. En déduire une fonction

`completion_max : graph -> graph`

calculant la complétion maximale d'un graphe.

4. Un graphe est dit *connexe* si pour tout couple de sommets du graphe x et y , il existe un chemin menant de x à y . On admettra la propriété suivante : un graphe est connexe si et seulement si pour tout couple de sommets x, y , sa complétion maximale contient une flèche allant de x à y . Ecrire une fonction **connexe** déterminant si un graphe est connexe par calcul puis examen de sa complétion maximale.